

Working with Data

Session 1 — What is this data?

Block 2 — One table, one question: SQL on DuckDB

This block in one sentence

A number is a sentence before it is a query: *a measure, over a population, after a filter.*

SQL is that sentence written precisely enough for a machine to answer it and for a colleague to check it.

NULL is waiting at every clause.

Agenda

1. Why SQL: 525,461 lines, watched
2. The map: the order a query runs
3. The number as a sentence · the aggregates · `NULL` in them
4. `GROUP BY`, the `NULL` group, the sum identity, a share of a total
5. `HAVING` against `WHERE`
6. Filters and `NULL`, mechanised · calculated columns · dates
7. Lab 2 — on the other file

Why SQL — watch

The whole Online Retail file, **525,461 invoice lines**, in DuckDB, run in the Lab 2 folder:

```
$ uv run python - <<'EOF'
import time, duckdb
con = duckdb.connect()
t = time.perf_counter()
lines = con.sql("SELECT COUNT(*) FROM 'data/raw/online_retail.parquet'").fetchone()[0]
by_country = con.sql("""
    SELECT Country, SUM(Quantity * Price) AS revenue
    FROM 'data/raw/online_retail.parquet'
    GROUP BY Country
    ORDER BY revenue DESC
""").df()
print(f"{lines:,} lines, {len(by_country)} countries, {time.perf_counter() - t:.3f} seconds")
EOF
525,461 lines, 40 countries, 0.182 seconds
```

captured on the build machine, 25 September 2026

The same file in Excel — watch

The same lines as a CSV: **46,810,498 bytes (46.8 MB)**. As Parquet: **3,218,445 bytes (3.2 MB)**. DuckDB reads either as a table.

```
$ wc -c online_retail_II_2009-2010.csv
46810498 online_retail_II_2009-2010.csv
$ head -3 online_retail_II_2009-2010.csv
Invoice,StockCode,Description,Quantity,InvoiceDate,Price,Customer ID,Country
489434,85048,15CM CHRISTMAS GLASS BALL 20 LIGHTS,12,2009-12-01 07:45:00,6.95,13085.0,United Kingdom
489434,79323P,PINK CHERRY LIGHTS,12,2009-12-01 07:45:00,6.75,13085.0,United Kingdom
```

525,461 rows is under Excel's documented limit of 1,048,576 rows a sheet. The question is not *can it*. It is:

- **Can a colleague check it?** A query is text they can read and rerun. A pivot is clicks.
- **Next month's file?** One line, rerun. The clicks, again.
- **What does it do with a mistake?** DuckDB refuses to turn text it cannot read into a number. A spreadsheet guesses.

Parquet, in one slide

	CSV	Parquet
the same 525,461 rows	46.8 MB	3.2 MB
stored	row after row, as text	column by column, compressed
types	guessed every time it is read	stored with the data

`Customer ID` is still a `DOUBLE` in the Parquet: the type the CSV reader guessed was saved with it. That is all this course says about Parquet.

The lecture file: one row is one country in one year

Watch (cell 1 of `lecture.ipynb`): Block 1's key test, on the lecture file.

cell 1

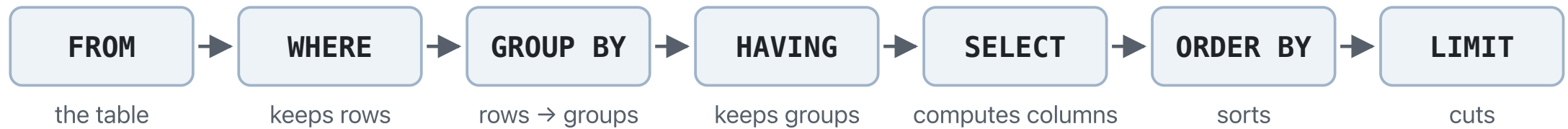
```
con.sql("""
    SELECT COUNT(*)                AS n_rows,
           COUNT(DISTINCT (country, year)) AS n_keys,
           COUNT(DISTINCT country)      AS countries
    FROM 'data/raw/countries.csv'
""").df()
```

	n_rows	n_keys	countries
0	7665	7665	219

219 countries, 1990 to 2024. One row per country per year?

The map: the order a query runs

The order it runs



The order you write it: `SELECT · FROM · WHERE · GROUP BY · HAVING · ORDER BY · LIMIT`

Each clause gets the result of the one before it. Nothing can use what has not been made yet.

You write **SELECT** first. It runs **fifth**.

The sentence first, then the query

Total CO₂ emitted by countries in 2024 is the sum of **co2** (the measure) **over the country rows** (the population) **of 2024** (the filter).

Predict, then watch (cell 2):

```
con.sql("""
    SELECT SUM(co2) AS total_co2
    FROM 'data/raw/countries.csv'
    WHERE year = 2024
""").df()
```

Predict: how many rows come back? One per country, or one in all?

The sentence first, then the query — what it printed

cell 2

```
con.sql("""  
    SELECT SUM(co2) AS total_co2  
    FROM 'data/raw/countries.csv'  
    WHERE year = 2024  
""").df()
```

	total_co2
0	37406.117

Seven aggregates, and what **NULL** does to them

Predict, then watch (cell 3). Predict: are **n_rows** and **n_co2** equal?

```
con.sql("""
    SELECT COUNT(*) AS n_rows, COUNT(co2) AS n_co2, COUNT(DISTINCT country) AS n_countries,
           SUM(co2) AS total_co2, AVG(co2) AS avg_co2, MIN(co2) AS min_co2, MAX(co2) AS max_co2
    FROM 'data/raw/countries.csv'
    WHERE year = 2024
""").df()
```

Seven aggregates, and what **NULL** does to them — what it printed

```
cell 3 con.sql("""  
    SELECT COUNT(*) AS n_rows, COUNT(co2) AS n_co2, COUNT(DISTINCT country) AS n_countries,  
           SUM(co2) AS total_co2, AVG(co2) AS avg_co2, MIN(co2) AS min_co2, MAX(co2) AS max_co2  
    FROM 'data/raw/countries.csv'  
    WHERE year = 2024  
""").df()
```

	n_rows	n_co2	n_countries	total_co2	avg_co2	min_co2	max_co2
0	219	216	219	37406.117	173.176468	0.0	12289.037

An average with the missing left out, and with them as zeros

Predict, then watch (cell 4):

```
con.sql("""
    SELECT AVG(gdp)                AS avg_gdp,
           AVG(COALESCE(gdp, 0)) AS avg_gdp_zero
    FROM 'data/raw/countries.csv'
    WHERE year = 2022
""").df()
```

`COALESCE(gdp, 0)` returns the first of its values that is not `NULL` : `gdp` where there is one, otherwise `0`.

Predict: the same number twice, or two different ones? If different, which is larger? (The output prints `e+11` : $\times 10^{11}$.)

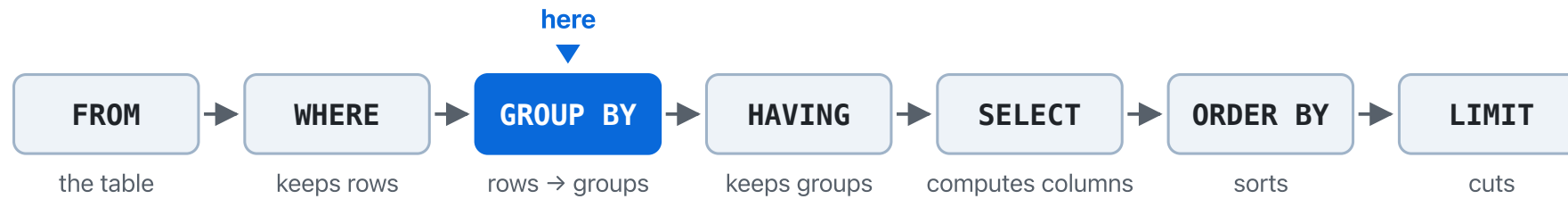
An average with the missing left out, and with them as zeros — what it printed

```
cell 4 con.sql("""  
    SELECT AVG(gdp) AS avg_gdp,  
           AVG(COALESCE(gdp, 0)) AS avg_gdp_zero  
    FROM 'data/raw/countries.csv'  
    WHERE year = 2022  
""").df()
```

	avg_gdp	avg_gdp_zero
0	7.943899e+11	5.948856e+11

GROUP BY: one row per year

The order it runs



The order you write it: SELECT · FROM · WHERE · GROUP BY · HAVING · ORDER BY · LIMIT

One row per value of the grouping column. Every other column must be aggregated.

Watch (cell 5). One row per year.

GROUP BY — what it printed

```
cell 5 con.sql("""
        SELECT year, SUM(co2) AS total_co2
        FROM 'data/raw/countries.csv'
        GROUP BY year
        ORDER BY year
        """).df()
```

	year	total_co2
0	1990	22184.276
1	1991	22662.918
2	1992	21966.603
3	1993	22192.438
4	1994	22389.768
5	1995	22899.373
6	1996	23564.971
7	1997	23677.035
8	1998	23549.459
9	1999	24064.355
10	2000	24727.963
11	2001	24915.493

	year	total_co2
12	2002	25468.479
13	2003	26823.432
14	2004	27706.560
15	2005	28659.728
16	2006	29603.555
17	2007	30450.998
18	2008	30991.476
19	2009	30498.959
20	2010	32245.651
21	2011	33373.932
22	2012	33860.648
23	2013	34165.627

	year	total_co2
24	2014	34349.848
25	2015	34250.914
26	2016	34223.754
27	2017	34773.495
28	2018	35497.626
29	2019	35846.991
30	2020	34318.680
31	2021	35987.784
32	2022	36562.105
33	2023	37011.294
34	2024	37406.117

The rule **GROUP BY** enforces

Predict, then watch (cell 6a; it is wrong on purpose). Predict: does DuckDB pick a country, or refuse?

```
SELECT year, country, SUM(co2) FROM 'data/raw/countries.csv' GROUP BY year
```

The rule **GROUP BY** enforces — what it printed

```
cell 6a  # The try/except prints DuckDB's refusal instead of stopping the notebook.  
try:  
    con.sql("""  
        SELECT year, country, SUM(co2)  
        FROM 'data/raw/countries.csv'  
        GROUP BY year  
        """).df()  
except duckdb.Error as e:  
    print(e)
```

Binder Error: column "country" must appear in the GROUP BY clause or must be part of an aggregate function.
Either add it to the GROUP BY list, or use "ANY_VALUE(country)" if the exact value of "country" is not important.

The fix: group by the identifier, aggregate the label

Watch (cell 6b).

cell 6b

```
# The grain stays one row per code; the name rides along inside any_value().
con.sql("""
    SELECT iso_code, any_value(country) AS country, SUM(co2) AS co2_since_1990
    FROM 'data/raw/countries.csv'
    GROUP BY iso_code
    ORDER BY co2_since_1990 DESC
    LIMIT 5
    """).df()
```

	iso_code	country	co2_since_1990
0	CHN	China	244175.556
1	USA	United States	192495.593
2	RUS	Russia	58797.248
3	IND	India	55646.011
4	JPN	Japan	41902.295

The **NULL** group

Predict, then watch (cell 7). Predict: how many rows — two, or three?

```
con.sql("""
    SELECT gdp > 1000000000000 AS big_economy, COUNT(*) AS countries, SUM(co2) AS co2
    FROM 'data/raw/countries.csv'
    WHERE year = 2022
    GROUP BY big_economy
    ORDER BY big_economy
""").df()
```

The **NULL** group — what it printed

cell 7

```
con.sql("""
    SELECT gdp > 1000000000000 AS big_economy, COUNT(*) AS countries, SUM(co2) AS co2
    FROM 'data/raw/countries.csv'
    WHERE year = 2022
    GROUP BY big_economy
    ORDER BY big_economy
""").df()
```

	big_economy	countries	co2
0	False	138	5611.665
1	True	26	30866.045
2	<NA>	55	84.395

The sum identity: a breakdown adds up to its total

Predict, then watch (cells 8a to 8c). Predict: do *big* plus *not big* add up to the total?

Emissions are **additive**: every breakdown adds up to the total, computed by other code, or one of them is wrong.

Compare sums with a tolerance, never `==`. Two correct sums of the same numbers can differ in the tenth decimal. Money: to the penny, `abs(a - b) < 0.005`.

Means and ratios have their own identities. An average does not add up. A ratio's top and bottom are reconciled separately, on the same rows.

The sum identity: a breakdown adds up to its total — what it printed

```
cell 8a total = con.sql("SELECT SUM(co2) FROM 'data/raw/countries.csv' WHERE year = 2022").fetchone()[0]
by_group = con.sql("""
    SELECT SUM(co2) FROM (
        SELECT gdp > 1000000000000 AS big_economy, SUM(co2) AS co2
        FROM 'data/raw/countries.csv'
        WHERE year = 2022
        GROUP BY big_economy
    )
""").fetchone()[0]
print(f"Total, 2022: {total:10,.3f} Mt")
print(f"The three groups, added up: {by_group:10,.3f} Mt")
```

Total, 2022:	36,562.105 Mt
The three groups, added up:	36,562.105 Mt

The two filters that sound like everything

Watch (cell 8b).

```
cell 8b big = con.sql("""SELECT SUM(co2) FROM 'data/raw/countries.csv'
                        WHERE year = 2022 AND gdp > 1000000000000""").fetchone()[0]
small = con.sql("""SELECT SUM(co2) FROM 'data/raw/countries.csv'
                  WHERE year = 2022 AND gdp <= 1000000000000""").fetchone()[0]

print(f"Total, 2022:                {total:10,.3f} Mt")
print(f"gdp > 1e12, plus gdp <= 1e12: {big + small:10,.3f} Mt")
print(f"missing from the two filters: {total - big - small:10,.3f} Mt")
```

Total, 2022:	36,562.105 Mt
gdp > 1e12, plus gdp <= 1e12:	36,477.710 Mt
missing from the two filters:	84.395 Mt

The same two sums, compared exactly

Watch (cell 8c).

```
cell 8c  print(f"Exactly: {total!r} against {by_group!r}")
         print("equal with == ?", total == by_group, "    within 0.0005 Mt ?", abs(total - by_group) < 0.0005)
```

```
Exactly: 36562.105000000001 against 36562.105
equal with == ? False    within 0.0005 Mt ? True
```

A share of a total: a query inside a query

Watch (cell 9). Each country's share of the countries' 2024 total:

```
cell 9 con.sql("""
    SELECT country, co2,
           ROUND(100 * co2 / (SELECT SUM(co2) FROM 'data/raw/countries.csv'
                               WHERE year = 2024), 2) AS pct_of_total
    FROM 'data/raw/countries.csv'
    WHERE year = 2024
    ORDER BY co2 DESC
    LIMIT 5
    """).df()
```

	country	co2	pct_of_total
0	China	12289.037	32.85
1	United States	4904.120	13.11
2	India	3193.478	8.54
3	Russia	1780.524	4.76
4	Japan	961.867	2.57

The inner query runs once and returns one number, 37,406.117. The outer query divides every row by it.

HAVING: a condition on a group

The order it runs



The order you write it: SELECT · FROM · WHERE · GROUP BY · HAVING · ORDER BY · LIMIT

A condition on a count or a sum can only be tested once the groups exist: HAVING.

Predict, then watch (cell 10a). The years in which fewer countries reported CO₂ than in 2024. Predict: could the condition go in **WHERE** instead of **HAVING** ?

```
con.sql("""
    SELECT year, COUNT(co2) AS reporting
    FROM 'data/raw/countries.csv'
    GROUP BY year
    HAVING COUNT(co2) < 216
    ORDER BY year
""").df()
```

HAVING: a condition on a group — what it printed

```
cell 10a # The prediction: could the condition go in WHERE? The try/except prints DuckDB's answer instead of stopping.
try:
    con.sql("""
        SELECT year, COUNT(co2) AS reporting
        FROM 'data/raw/countries.csv'
        WHERE COUNT(co2) < 216
        GROUP BY year
    """).df()
except duckdb.Error as e:
    print("In WHERE, DuckDB refused:", str(e).splitlines()[0])
```

In WHERE, DuckDB refused: Binder Error: WHERE clause cannot contain aggregates!

The same condition in **HAVING**

Watch (cell 10b).

cell 10b

```
con.sql("""  
    SELECT year, COUNT(co2) AS reporting  
    FROM 'data/raw/countries.csv'  
    GROUP BY year  
    HAVING COUNT(co2) < 216  
    ORDER BY year  
""").df()
```

	year	reporting
0	1990	209
1	1991	210
2	1992	212
3	1993	212

AND, OR, and the parentheses you forgot

Predict, then watch (cell 11). The sentence: *China and India, in 2024*. Predict: how many rows does each line return?

```
WHERE year = 2024 AND country = 'China' OR country = 'India'  
WHERE year = 2024 AND (country = 'China' OR country = 'India')
```

Three more tools for WHERE, one line each: country IN ('China', 'India') · year BETWEEN 2020 AND 2024 (both ends included) · country LIKE 'United%' (% is any text).

AND, OR, and the parentheses you forgot — what it printed

```
cell 11 without = con.sql("""
        SELECT COUNT(*) FROM 'data/raw/countries.csv'
        WHERE year = 2024 AND country = 'China' OR country = 'India'
        """).fetchone()[0]
with_parentheses = con.sql("""
        SELECT COUNT(*) FROM 'data/raw/countries.csv'
        WHERE year = 2024 AND (country = 'China' OR country = 'India')
        """).fetchone()[0]
print("without parentheses:", without)
print("with parentheses:    ", with_parentheses)
```

```
without parentheses: 36
with parentheses:    2
```

Every country but China

Predict, then watch (cell 12). 219 countries in 2024, minus China, written two ways. Predict both counts.

```
print(con.sql("""SELECT COUNT(*) FROM 'data/raw/countries.csv'
                WHERE year = 2024 AND iso_code != 'CHN'""").fetchone()[0])
print(con.sql("""SELECT COUNT(*) FROM 'data/raw/countries.csv'
                WHERE year = 2024 AND iso_code IS DISTINCT FROM 'CHN'""").fetchone()[0])
```

Every country but China — what it printed

```
cell 12 print(con.sql("""SELECT COUNT(*) FROM 'data/raw/countries.csv'
                        WHERE year = 2024 AND iso_code != 'CHN'""").fetchone()[0])
print(con.sql("""SELECT COUNT(*) FROM 'data/raw/countries.csv'
                WHERE year = 2024 AND iso_code IS DISTINCT FROM 'CHN'""").fetchone()[0])
```

217

218

NULL in a WHERE, mechanised — watch

For a row whose `iso_code` is `NULL` (Kosovo, in this table):

The condition	Its value	Kept by WHERE ?
<code>iso_code = 'CHN'</code>	<code>NULL</code> (unknown)	no
<code>iso_code != 'CHN'</code>	<code>NULL</code> (unknown)	no
<code>iso_code NOT IN ('CHN', 'IND')</code>	<code>NULL</code> (unknown)	no
<code>iso_code IS NULL</code>	true	yes
<code>iso_code IS DISTINCT FROM 'CHN'</code>	true	yes
<code>COALESCE(iso_code, 'none') != 'CHN'</code>	true	yes

WHERE keeps a row only when the condition is **true**. Unknown is not true.

Calculated columns, aliases, and division

Watch (cell 13b). `co2` is in million tonnes and `population` in persons, so tonnes per person is `co2 * 1000000 / population`:

cell 13b

```
con.sql("""
    SELECT country, co2, population,
           ROUND(co2 * 1000000 / population, 2) AS tonnes_per_person
    FROM 'data/raw/countries.csv'
    WHERE year = 2024
    ORDER BY co2 DESC
    LIMIT 5
""").df()
```

	country	co2	population	tonnes_per_person
0	China	12289.037	1419321284	8.66
1	United States	4904.120	345426566	14.20
2	India	3193.478	1450935785	2.20
3	Russia	1780.524	144820421	12.29
4	Japan	961.867	123753041	7.77

Division, and the name **AS** gives

Watch (cell 13a).

cell 13a `con.sql("SELECT 7 / 2 AS slash, 7 // 2 AS double_slash, ROUND(2 / 3, 2) AS rounded").df()`

	slash	double_slash	rounded
0	3.5	3	0.67

AS names a column. The name is born in **SELECT**, after **WHERE** on the map, so standard SQL does not let **WHERE** use it. DuckDB does, as a shortcut. This course writes the expression again: the query then means the same in any database.

Dates, the minimum

Watch (cell 14). One timestamp, three ways:

cell 14

```
con.sql("""
    SELECT date_trunc('month', TIMESTAMP '2010-03-17 14:05:00') AS month_start,
           year(TIMESTAMP '2010-03-17 14:05:00') AS the_year,
           strftime(TIMESTAMP '2010-03-17 14:05:00', '%Y-%m') AS label
""").df()
```

	month_start	the_year	label
0	2010-03-01	2010	2010-03

`date_trunc` turns a moment into the start of its month (or `'week'`, `'year'`): a value you can `GROUP BY`.

A period made from a number

Watch (cell 15). `year // 10 * 10 AS decade :`

```
cell 15 con.sql("""  
    SELECT year // 10 * 10 AS decade,  
           COUNT(DISTINCT year) AS years,  
           SUM(co2) AS total_co2  
    FROM 'data/raw/countries.csv'  
    GROUP BY decade  
    ORDER BY decade  
""").df()
```

	decade	years	total_co2
0	1990	10	229151.196
1	2000	10	279846.643
2	2010	10	342588.486
3	2020	5	181285.980

Build one clause at a time — watch

1. **The sentence:** the measure, the population, the filter.
2. `FROM` + `LIMIT 10` : look at ten rows. Are they the thing?
3. `WHERE` : **count the rows** it keeps. The number you expected?
4. `GROUP BY` : **say how many groups** before you run it.
5. The aggregates. Then `HAVING` , `ORDER BY` . Remove the `LIMIT` .
6. **The check:** the identity this number permits.

What you just saw, as one idea

A filter that sounds right can throw away rows the sentence keeps.

`gdp <= 1e12` and `gdp > 1e12` lost 55 countries. `!= 'CHN'` lost Kosovo.

A ratio can be built from two different populations.

`SUM(gdp) / COUNT(*)` is a top from 164 countries over a bottom of 219: 595 billion, the same wrong number as counting the missing as zero.

No error in either. The sentence says what the number counts; the query must do all of it, and only it.

Git thread — the message names the cause

```
git commit -m "Keep countries with no GDP in the breakdown: gdp <= 1e12 dropped 55 NULL rows"
```

not `git commit -m "fix"`. Commit after each fix. Six months from now, `git log --oneline` is the only diagnosis note anyone will read.

AI thread — the assistant does not know your data

Ask an assistant: *"Average GDP per country in 2022, from this table"*, and paste `DESCRIBE`. It will write `AVG(gdp)`. Then ask it: *"over how many countries?"*

It cannot know. `COUNT(gdp)` against `COUNT(*)`: 164 against 219.

The assistant does not know your data's conventions unless you paste them. The census is how you learn them yourself.

5-minute stand-and-stretch — start the Lab 2 clone first

In your terminal (Git Bash on Windows, Terminal on macOS), in the folder where you keep course work:

```
git clone https://github.com/earino/ecbs5294-lab02-one-table.git  
cd ecbs5294-lab02-one-table  
uv sync
```

Start it, then stand up: it downloads while you stretch.

Lab 2 — The revenue the brief did not ask for

The file: `data/raw/online_retail.parquet` — the shop's invoice lines. **The brief:** finance's definitions of every number, in the README and the notebook.

The symptom: the report runs without an error and says 2010 revenue was **£8,739,637.52** and the average identified customer brought in **£2,052.52**. Its own per-customer table does not average to £2,052.52. And every number has a sentence in the brief.

First minute: the clone finished? Open the folder in VS Code, open `notebooks/report.ipynb`, pick the `.venv` kernel, run the first cell: it prints the Lab 2 folder.

Your work, in `report.ipynb`: inspect (A) → a view `sales_2010` (B) → revenue by month, from an empty cell (C) → countries with more than 100 invoices, from an empty cell (D) → **the check** (E) → **DIAGNOSIS.md**. After the note, if there is time: the per-customer figure (F, also HW1 question 7). Neighbour at minute 33. Checkpoint on Moodle.

The last ten minutes — explain it to a neighbour

One of you explains, about a minute, pointing at the screen: **what was wrong · why · the query that proved it · what you changed · how you know it is right.** The other asks:

1. Show me the query that proves it.
2. Why was it wrong, not just where?
3. A colleague says your view's filter is the long way round: ***WHERE Quantity > 0** does the same job.* Same lines? Same revenue?

Then swap. Unsure, or you disagree? **Hands up.** Then the checkpoint on Moodle.

Homework 1 assigned — One table, the weekly numbers

The head of sales wants **seven numbers** from the same file, each **a sentence, a query, a number, and a check**; two more are **stretch**, not graded. The brief carries Lab 2's definitions and a few more. Expected time: **3–4 hours**. Due **Friday 9 October, 23:59**, on Moodle.

The starter is `earino/ecbs5294-hw01-one-table` on GitHub; its README has the clone line, for home.

1. The notebook: every question in its four parts, and the **reconciliation** — by month = by country = the total, to the penny
2. **The trap log** in `DIAGNOSIS.md` : one five-part note per data issue that changed a number, even if your query was never wrong
3. Commits that name the cause · `GIT_LOG.txt` · the zip by `git archive` , as `SUBMITTING.md` says
4. A **60–90 second video**: one trap — symptom, cause, change, verification
5. `AI_USE.md`